Révision contrôle — Les fonctions PL/pgSQL

R3.07 · SQL dans un langage de programmation · BUT Informatique 2

Fiche de révision + exercices corrigés (base bdd_car_net)

14 octobre 2026

Sommaire

- Comment utiliser ce document
- Partie 1 — Fiche de cours
  - 1.1 Le squelette d'une fonction (à connaître par cœur)
  - 1.2 Les variables
  - 1.3 Les paramètres
  - 1.4 L'affectation : 4 façons de mettre une valeur dans une variable
  - 1.5 Structures de contrôle
  - 1.6 Le retour d'une fonction — LE tableau à maîtriser
  - 1.7 Messages et erreurs
  - 1.8 Les curseurs
  - 1.9 EXECUTE : le SQL dynamique
  - 1.10 Fonctions SQL à avoir sous la main
  - 1.11 Triggers (rappel, au cas où)
- Partie 2 — Pièges et erreurs classiques
- Partie 3 — Méthode au contrôle
  - 3.1 Avant de commencer (5 minutes)
  - 3.2 Le patron de réponse pour chaque fonction
  - 3.3 Grille mentale « quel RETURNS ? »
- Partie 4 — Exercices d'entraînement (base `bdd_car_net`)
  - 4.1 Niveau 1 — fonctions scalaires
  - 4.2 Niveau 2 — plusieurs colonnes, plusieurs lignes
  - 4.3 Niveau 3 — curseurs, mises à jour, niveau contrôle
  - 4.4 Le contrôle d'entraînement, à refaire en conditions réelles (1h30)
- Corrections
  - Réponses de « trouve les erreurs » (partie 2)
  - Corrections des exercices 1 à 17
  - Correction du contrôle d'entraînement (version du prof, commentée)
- Mémo d'une page (à relire juste avant d'entrer)

Comment utiliser ce document

Le contrôle dure **1h30, sur machine, sans internet**, avec droit au **cours** et aux **corrections de TP**. Il porte sur les **fonctions** (et très probablement les curseurs, qui sont dans le même cours ; les triggers sont un bonus peu probable). Le contrôle d'entraînement du prof utilise la base `bdd_car_net` : il y a de fortes chances que le vrai contrôle utilise la même base ou une base très proche, avec le même type de questions.

Ce document est en 4 parties :

1. **Fiche de cours** : tout ce qu'il faut savoir, condensé, avec la syntaxe exacte du prof.
2. **Pièges et erreurs classiques** : les erreurs qui coûtent des points (et du temps).
3. **Méthode au contrôle** : le patron de réponse à appliquer à chaque exercice.
4. **Exercices d'entraînement** sur `bdd_car_net`, du plus simple au niveau contrôle, puis le contrôle d'entraînement refait, puis les **corrections** (toutes testées sur la base, les résultats indiqués sont réels).

Conseil de planning pour les 3 heures : 45 min de cours (parties 1 à 3), 1h30 d'exercices en les faisant vraiment dans phppgadmin, 30 min pour refaire le contrôle d'entraînement en conditions réelles.

Partie 1 — Fiche de cours

1.1 Le squelette d'une fonction (à connaître par cœur)

```
CREATE OR REPLACE FUNCTION nom_fonction(p_param1 type, p_param2 type) RETURNS type_retour
        AS $$
DECLARE
    -- déclarations de variables (une par ligne, terminées par ;)
    v_variable  type;
BEGIN
    -- instructions, chacune terminée par ;
    RETURN valeur;
END;
$$ LANGUAGE plpgsql;
```

Points clés :

- Les `$$` délimitent le corps de la fonction. Sans eux, le moteur SQL prend le premier `;` pour la fin de la requête → erreur (vidéo « erreur délimiteur »).
- `CREATE OR REPLACE` permet de recréer la fonction sans la supprimer… **sauf si on change le type de retour** (voir pièges).
- `LANGUAGE plpgsql` en fin, obligatoire.
- Chaque déclaration et chaque instruction se termine par `;`. Les déclarations multiples `i, j integer;` sont **interdites**.
- Appel d'une fonction scalaire : `SELECT nom_fonction(valeurs);`
- Commentaires : `-- une ligne` ou `/* plusieurs lignes */`. Le prof les considère quasi obligatoires : commentez vos vérifications.

Exemple du prof (TP1) :

```
CREATE OR REPLACE FUNCTION moySalaire() RETURNS NUMERIC AS $$
DECLARE
    V_Moyenne numeric;
BEGIN
    SELECT AVG(salaire) INTO V_Moyenne FROM employes;
    RETURN V_Moyenne;
END
$$ LANGUAGE plpgsql;

SELECT moySalaire();
SELECT nom, salaire FROM employes WHERE salaire > moySalaire();  -- utilisable dans une
        requête
```

1.2 Les variables

```
DECLARE
    age         integer;                    -- simple
    ok          boolean := true;            -- avec valeur initiale
```

```
    quantite     integer DEFAULT 32;           -- idem avec DEFAULT
    taux         CONSTANT numeric := 0.2;      -- constante, non modifiable
    v_prix       vehicule.prix_jour%TYPE;      -- même type qu'une colonne
    v_cli        client%ROWTYPE;               -- une ligne entière de la table client
    v_ligne      RECORD;                       -- ligne sans structure prédéfinie
    somme        real;
```

Type	Usage	Accès aux champs
`table.colonne%TYPE`	variable du même type qu'une colonne	—
`table%ROWTYPE`	une ligne complète d'une table (toutes les colonnes)	`v_cli.nom_client`
`RECORD`	ligne de structure libre, prend la structure du `SELECT` ou du `FOR` qui la remplit	`v_ligne.exemple`

Un `RECORD` n'a **pas de structure tant qu'il n'a pas été assigné** : accéder à un champ avant provoque une erreur d'exécution.

Identificateurs : commencent par une lettre, lettres/chiffres/`_`, **insensibles à la casse** (`SommePaye` = `sommepaye`). Convention du prof : `p_` pour les paramètres, `v_` pour les variables (ça évite les conflits de noms avec les colonnes !).

1.3 Les paramètres

```
-- paramètres nommés (le plus courant)
CREATE FUNCTION add_em(x integer, y integer) RETURNS integer AS $$
BEGIN RETURN x + y; END; $$ LANGUAGE plpgsql;

-- paramètres anonymes : $1, $2 ...
CREATE FUNCTION add_em(integer, integer) RETURNS integer AS $$
BEGIN RETURN $1 + $2; END; $$ LANGUAGE plpgsql;

-- alias (ancienne méthode)
DECLARE
    prixHT ALIAS FOR $1;
    taux   ALIAS FOR $2;

-- valeurs par défaut (seulement en fin de liste)
CREATE FUNCTION somme(x numeric, y numeric DEFAULT 2, z numeric DEFAULT 2) RETURNS
        numeric ...
SELECT somme(3,5,6);  -- 14
SELECT somme(3);      -- 7

-- paramètres de sortie OUT : pas de RETURNS, pas de RETURN
CREATE OR REPLACE FUNCTION somme(IN x numeric, IN y numeric, OUT somme numeric) AS $$
BEGIN
    somme := x + y;
END; $$ LANGUAGE plpgsql;
SELECT somme(4,7);
```

Deux avertissements du prof :

- **Pas de polymorphisme** (note de cours) : deux fonctions de même nom avec des paramètres différents coexistent et font des choses différentes → à éviter catégoriquement. Si vous changez les paramètres d'une fonction, faites `DROP FUNCTION ancien_nom(anciens types);` avant.
- **Attention au type des arguments** (vidéo « erreur type arg ») : `varchar(45)` vs `text`, `int` vs `numeric`… Déclarez un type compatible avec la colonne que vous comparez. Si l'appel échoue avec « function xxx(unknown) does not exist » ou « does not match », c'est un problème de type.

1.4 L'affectation : 4 façons de mettre une valeur dans une variable

```
-- 1. affectation directe
v_date := '2004-10-10';      -- ou  v_date = '2004-10-10';

-- 2. SELECT ... INTO  (requête qui renvoie UNE ligne, une ou plusieurs colonnes)
SELECT prix_jour INTO v_prix FROM vehicule WHERE code_vehicule = p_code;
SELECT nom_client, pre_client INTO v_nom, v_prenom FROM client WHERE num_client = p_num;
SELECT nom_client, pre_client INTO v_rec FROM client WHERE num_client = p_num;  -- dans
        un record
-- forme équivalente du prof :  SELECT INTO v_moyenne AVG(salaire) FROM employes;

-- 3. variable := (requête)   (requête à UNE colonne et UNE ligne)
v_nb := (SELECT count(*) FROM reservation WHERE num_client = p_num);

-- 4. requête dans un test
IF (SELECT statut FROM reservations WHERE id_reservation = p_id) = 'ANNULEE' THEN ...
```

Règle d'or : un SELECT dans une fonction doit TOUJOURS avoir une destination (`INTO`, affectation, test, ou `FOR ... IN`). Un `SELECT` « nu » donne l'erreur `query has no destination for result data`.

PERFORM et FOUND — la vérification d'existence (dans TOUS les exercices)

`PERFORM` = un `SELECT` dont on ignore le résultat. Il positionne la variable spéciale `FOUND` : vraie si la requête a ramené au moins une ligne.

```
PERFORM * FROM agence WHERE lower(nom_agence) = lower(p_nom_ag);
IF NOT FOUND THEN
    RAISE EXCEPTION 'l''agence % n''existe pas', p_nom_ag;
END IF;
```

`FOUND` est aussi positionné par `SELECT INTO`, `UPDATE`, `INSERT`, `DELETE`, `FETCH`, `FOR`. Donc après un `SELECT ... INTO`, on peut aussi tester `IF NOT FOUND`.

1.5 Structures de contrôle

```
-- IF
IF cond THEN ...
ELSIF cond2 THEN ...
ELSE ...
END IF;
```

```plpgsql
-- CASE
CASE x
    WHEN 1, 2 THEN msg := 'un ou deux';
    ELSE msg := 'autre';
END CASE;

-- LOOP (sortie obligatoire avec EXIT ou RETURN)
LOOP
    ...
    EXIT WHEN compteur > 10;      -- ou : IF compteur > 10 THEN EXIT; END IF;
END LOOP;

-- WHILE
WHILE montant > 0 LOOP
    ...   -- faire évoluer la variable testée !
END LOOP;

-- FOR sur un intervalle d'entiers
FOR i IN 1..10 LOOP
    RAISE NOTICE 'i vaut %', i;
END LOOP;
FOR i IN REVERSE 10..1 LOOP ... END LOOP;

-- FOR sur le résultat d'une requête  ***  LE PLUS IMPORTANT  ***
FOR v_ligne IN SELECT exemple, prix_jour FROM vehicule WHERE categorie = p_cat   -- PAS
        de ; ici
LOOP
    RETURN NEXT v_ligne;
END LOOP;

-- FOR sur une requête dynamique
FOR v_ligne IN EXECUTE 'SELECT * FROM ' || quote_ident(p_table) LOOP ... END LOOP;
```

Boucles imbriquées avec label : `<<loop1>> LOOP ... EXIT loop1 WHEN y > 4; ... END LOOP;`.

1.6 Le retour d'une fonction — LE tableau à maîtriser

C'est le cœur du contrôle. Le choix dépend du nombre de **lignes** et de **colonnes** renvoyées.

Résultat	RETURNS	Instruction de retour	Appel
1 ligne, 1 colonne (scalaire)	`RETURNS integer` / `numeric` / `varchar` / `boolean` …	`RETURN valeur;`	`SELECT f(…);`
1 ligne, N colonnes	`RETURNS record`	`RETURN v_record;`	`SELECT * FROM f(…) AS (c1 type, c2 type…);`
N lignes, 1 colonne	`RETURNS SETOF type`	`RETURN NEXT valeur;` dans une boucle puis `RETURN;`	`SELECT f(…);` ou `SELECT * FROM f(…);`
N lignes, N colonnes	`RETURNS SETOF record`	`RETURN NEXT v_record;` dans une boucle puis `RETURN;`	`SELECT * FROM f(…) AS (c1 type, c2 type…);`

Résultat	RETURNS	Instruction de retour	Appel
N lignes = lignes complètes d'une table	`RETURNS SETOF nom_table`	`RETURN NEXT v_row;` (où `v_row` est `table%ROWTYPE`) ou `RETURN QUERY SELECT …;`	`SELECT * FROM f(…);` (pas de `AS`)
N lignes, colonnes nommées à l'avance	`RETURNS TABLE(c1 type, c2 type…)`	`RETURN QUERY SELECT …;`	`SELECT * FROM f(…);` (pas de `AS`)
rien	`RETURNS void`	`RETURN;` facultatif	`SELECT f(…);`

Règle de la note de cours, à retenir mot pour mot :

> Si ça retourne **plus d'une colonne** : `SELECT * FROM f(12) AS (col1 type, col2 type, ...)`. Si ça retourne **plus d'une ligne** (et une seule colonne) : `SELECT * FROM f(12)` ou `SELECT f(12)`.

Dans le `AS (...)`, les **noms** sont libres, mais les **types doivent correspondre** aux colonnes renvoyées (`varchar`, `numeric`, `int`, `date`, `time`, `boolean`, `bigint` pour un `count(*)` …). On peut ensuite n'afficher qu'une colonne : `SELECT n FROM info(1) AS (n varchar, s real);`.

Les 4 modèles à recopier

Scalaire (1 ligne, 1 colonne) :

```
CREATE OR REPLACE FUNCTION SommePaye(p_numCli int) RETURNS real AS $$
DECLARE
    somme real;
BEGIN
    PERFORM num_client FROM client WHERE num_client = p_numCli;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'le client % n''existe pas', p_numCli;
    END IF;
    SELECT sum(prix_jour * extract(day FROM (date_heure_retour - date_heure_depart)))
    INTO somme
    FROM reservation INNER JOIN vehicule USING (code_vehicule)
    WHERE num_client = p_numCli;
    RETURN somme;
END
$$ LANGUAGE plpgsql;

SELECT SommePaye(100);   -- 5129
```

Record (1 ligne, N colonnes) :

```
CREATE OR REPLACE FUNCTION sup(P_NumEmp integer) RETURNS RECORD AS $$
DECLARE
    Var_NomSup RECORD;
BEGIN
    SELECT nom, prenom, nosupr INTO Var_NomSup FROM employes
    WHERE noemp = (SELECT nosupr FROM employes WHERE noemp = P_NumEmp);
    RETURN Var_NomSup;
```

```
END;
$$ LANGUAGE plpgsql;

SELECT * FROM sup(17) AS (nom VARCHAR, prenom VARCHAR, nosupr NUMERIC(7,0));
```

SETOF record (N lignes, N colonnes) — le modèle le plus demandé :

```
CREATE OR REPLACE FUNCTION listeVehicule(p_nom_ag varchar(45)) RETURNS SETOF record AS $$
DECLARE
    v_ligne record;
BEGIN
    -- vérification de l'existence de l'agence
    PERFORM nom_agence FROM agence WHERE lower(nom_agence) = lower(p_nom_ag);
    IF NOT FOUND THEN
        RAISE EXCEPTION 'l''agence % n''existe pas', p_nom_ag;
    END IF;
    -- parcours du résultat et renvoi ligne par ligne
    FOR v_ligne IN SELECT exemple, prix_jour
                   FROM vehicule INNER JOIN proposer USING (code_vehicule)
                                 INNER JOIN agence USING (num_agence)
                   WHERE lower(nom_agence) = lower(p_nom_ag)
    LOOP
        RETURN NEXT v_ligne;
    END LOOP;
    RETURN;
END
$$ LANGUAGE plpgsql;

SELECT * FROM listeVehicule('Micsot') AS (nom_vehi varchar, px numeric);
```

SETOF table + RETURN QUERY (lignes complètes d'une table) :

```
CREATE OR REPLACE FUNCTION voyages_par_guide(p_guide INTEGER) RETURNS SETOF voyages AS $$
BEGIN
    PERFORM * FROM guides WHERE id_guide = p_guide;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'le guide d''id % n''existe pas', p_guide;
    END IF;
    RETURN QUERY SELECT voyages.* FROM voyages WHERE id_guide = p_guide;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM voyages_par_guide(1);
```

1.7 Messages et erreurs

```
RAISE NOTICE 'taux % et prix = %', taux, prix;      -- message d'info, n'arrête pas
RAISE EXCEPTION 'le client % n''existe pas', p_num; -- arrête la fonction et annule la
        transaction
```

- Niveaux : `DEBUG, LOG, INFO, NOTICE, WARNING, EXCEPTION`.
- Chaque `%` est remplacé par l'expression correspondante, dans l'ordre.
- Une apostrophe dans le message s'écrit **doublée** : `'n''existe pas'`.

Bloc de récupération d'erreur (moins fréquent au contrôle mais au programme) :

```
BEGIN
    y := x / p_diviseur;
    INSERT INTO formateur VALUES (3, 'Morel', 'Paul', 45);
EXCEPTION
    WHEN division_by_zero THEN
        RAISE EXCEPTION 'Erreur division_by_zero';
    WHEN unique_violation THEN
        UPDATE formateur SET (nom_form, prenom_form, nbh) = ('Morel', 'Paul', 45) WHERE
        id_formateur = 3;
END;
```

Autres conditions utiles : `no_data_found`, `too_many_rows`, `foreign_key_violation`, `check_violation`, `OTHERS`.

1.8 Les curseurs

Un curseur = une requête « stockée » dont on parcourt le résultat ligne par ligne. C'est l'équivalent du `FOR ... IN requête`, mais explicite (le prof demande parfois expressément un curseur).

```
DECLARE
    -- 3 formes de déclaration
    curs1 refcursor;                                        -- non lié : requête
        donnée à l'OPEN
    curs2 CURSOR FOR SELECT * FROM client WHERE type_client = 'Fidéle';   -- lié
    curs3 CURSOR (key integer) IS SELECT * FROM reservation WHERE num_client = key;  --
        lié et paramétré
    v_ligne record;
BEGIN
    OPEN curs2;                    -- ouvre et exécute la requête
    OPEN curs3(p_num);             -- curseur paramétré
    OPEN curs1 FOR SELECT * FROM agence WHERE pays_agence = p_pays;   -- non lié
    OPEN curs1 FOR EXECUTE 'SELECT * FROM ' || quote_ident(p_table);

    LOOP
        FETCH curs3 INTO v_ligne;      -- lit la ligne courante
        EXIT WHEN NOT FOUND;           -- plus de ligne → on sort
        ... traitement, par ex. RETURN NEXT v_ligne; ...
    END LOOP;

    CLOSE curs3;                   -- toujours fermer
```

`CURSOR FOR` et `CURSOR IS` sont équivalents. Modèle complet (correction TP1) :

```
CREATE OR REPLACE FUNCTION collegues_curs(P_NumEmp integer) RETURNS SETOF RECORD AS $$
DECLARE
    V_rec RECORD;
    curs1 CURSOR(key integer) IS SELECT prenom, e.nom
                                 FROM employes e INNER JOIN departement d ON e.nodept =
        d.nodept
                                 WHERE d.nom = departement(key) AND noemp != key;
BEGIN
    OPEN curs1(P_NumEmp);
    LOOP
        FETCH curs1 INTO V_rec;
        EXIT WHEN NOT FOUND;
```

```
        RETURN NEXT V_rec;
    END LOOP;
    CLOSE curs1;
    RETURN;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM collegues_curs(7) AS (prenom VARCHAR, nom VARCHAR);
```

1.9 EXECUTE : le SQL dynamique

Indispensable quand un **nom de table ou de colonne** vient d'un paramètre (on ne peut pas écrire `FROM p_table`).

```
EXECUTE 'CREATE TABLE ' || quote_ident(p_temp) || '(id_f integer, nom varchar(50))';
EXECUTE 'INSERT INTO ' || quote_ident(p_temp) || ' VALUES(' || use_form.id_formateur ||
        ', '
        || quote_literal(use_form.nom_formateur) || ')';
EXECUTE 'UPDATE tbl SET ' || quote_ident(colname) || ' = ' || quote_literal(p_name) || '
        WHERE ...';
```

- `quote_ident(x)` → identifiant (table, colonne) correctement protégé par des guillemets doubles.
- `quote_literal(x)` → valeur chaîne entourée d'apostrophes, caractères spéciaux échappés.
- Pensez aux espaces dans la concaténation (`' WHERE '` et non `'WHERE'`).

1.10 Fonctions SQL à avoir sous la main

Besoin	Écriture	Remarque
nombre de jours entre deux `TIMESTAMP`	`extract(day FROM (date_heure_retour - date_heure_depart))`	**le prof insiste** : on ne garde que les jours de l'intervalle
nombre de jours entre deux `DATE`	`date_fin - date_deb`	donne directement un entier
âge	`extract(year FROM age(current_date, date_naissance))`	
remplacer NULL	`coalesce(pts_bonus_acquis, 0)`	indispensable avant une addition ou un `sum` qui peut être vide
arrondir	`round(x, 2)` ou `x::numeric(10,2)`	
conversion	`x::int`, `x::real`, `x::text`, `CAST(x AS int)`	
comparer sans la casse	`lower(nom_agence) = lower(p_nom)`	le prof le fait pour les noms d'agence
concaténer	`nom || ' ' || prenom`	

Besoin	Écriture	Remarque
premier d’un classement	`ORDER BY … DESC LIMIT 1`	utilisé pour « la tranche la plus haute ≤ nb_jours »
max d’une moyenne par groupe	`HAVING avg(note) >= ALL (SELECT avg(note) FROM … GROUP BY …)`	correction TP2
nombre de lignes modifiées par un UPDATE	`GET DIAGNOSTICS v_nb = ROW_COUNT;`	
date du jour	`current_date`, `current_timestamp`, `now()`	
nombre de résas par groupe	`count(*) … GROUP BY` ; en `AS (...)` le type est `bigint`	

1.11 Triggers (rappel, au cas où)

```
-- 1. la fonction trigger : sans paramètre, RETURNS trigger, renvoie NEW (ou NULL pour
        annuler en BEFORE)
CREATE OR REPLACE FUNCTION emp_stamp() RETURNS trigger AS $$
BEGIN
    IF NEW.salaire IS NULL THEN
        RAISE EXCEPTION '% doit avoir un salaire', NEW.nom_employe;
    END IF;
    NEW.date_dermodif := current_timestamp;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

-- 2. le trigger lui-même (TOUJOURS après la fonction)
CREATE TRIGGER emp_stamp BEFORE INSERT OR UPDATE ON emp
FOR EACH ROW EXECUTE PROCEDURE emp_stamp();
```

Variables spéciales : `NEW` (ligne après INSERT/UPDATE), `OLD` (ligne avant UPDATE/DELETE), `TG_OP` (`'INSERT'`, `'UPDATE'`, `'DELETE'`), `TG_TABLE_NAME`. `FOR EACH ROW` obligatoire pour utiliser `NEW`/`OLD`. Un trigger `BEFORE` qui renvoie `NULL` annule l’opération pour cette ligne.

Partie 2 — Pièges et erreurs classiques

Les 5 vidéos « erreur à ne pas faire » du cours, plus celles vues en TP :

#	Erreur	Message PostgreSQL	Correction
1	Oubli des `$$`	`syntax error at or near "DECLARE"` ou erreur au premier `;`	Entourer le corps avec `AS $$ ... $$ LANGUAGE plpgsql;`
2	Type de paramètre incompatible avec la colonne	`operator does not exist: character varying = integer` / `function … does not exist`	Mettre le même type que la colonne (ou caster)
3	`SELECT` sans destination	`query has no destination for result data`	Ajouter `INTO v_var`, ou utiliser `PERFORM` si on ignore le résultat
4	Nom de table/colonne en paramètre utilisé directement	`relation "p_table" does not exist`	Passer par `EXECUTE 'SELECT … ' || quote_ident(p_table)`
5	Appel d'une fonction `record` sans `AS (...)`	`a column definition list is required for functions returning "record"`	`SELECT * FROM f() AS (c1 type, c2 type)`
6	Appel d'un `SETOF record` comme scalaire `SELECT f()`	`materialize mode required, but it is not allowed in this context`	`SELECT * FROM f() AS (...)`
7	Changer le type de retour avec `CREATE OR REPLACE`	`cannot change return type of existing function`	`DROP FUNCTION f(types);` puis recréer
8	Mauvais types dans le `AS (...)`	`structure of query does not match function result type` / erreur de cast	Vérifier chaque type (ex. `count(*)` → `bigint`, `prix_jour` → `numeric`, `heure_ouverture` → `time`)
9	`SELECT INTO` qui renvoie plusieurs lignes	seule la première est prise, sans erreur → résultat faux	Ajouter une condition, `LIMIT 1`, ou utiliser une boucle `FOR`
10	Point-virgule après la requête d'un `FOR … IN requête`	`syntax error`	`FOR r IN SELECT … LOOP` (pas de `;` avant `LOOP`)

#	Erreur	Message PostgreSQL	Correction
11	Oubli du `RETURN;` final après les `RETURN NEXT`	fonctionne souvent, mais le prof l'exige dans ses corrections	Toujours finir par `RETURN;`
12	Apostrophe non doublée dans un message	`syntax error`	`'n''existe pas'`
13	Nom de variable identique à un nom de colonne	`column reference "nom" is ambiguous`	Préfixer `p_` / `v_`
14	Valeur texte comparée avec une mauvaise orthographe	aucune erreur, juste `FOUND` faux partout	Vérifier avec `SELECT DISTINCT type_client FROM client;` (voir ci-dessous)
15	Oubli de `SET search_path`	`relation "client" does not exist`	`SET search_path = bdd_car_net;` en début de session, ou préfixer les tables
16	`sum()` sur zéro ligne renvoie NULL	la fonction renvoie NULL	`coalesce(sum(...), 0)`
17	Division entière	`7/2 = 3`	caster : `7::numeric / 2`

Piège spécifique à `bdd_car_net` : les valeurs stockées sont `'Fidéle'` (avec un **é** et non è) et `'Occassionnel'` (avec **deux s**). C'est d'ailleurs ce que le prof écrit dans sa correction. Au contrôle, la **première chose à faire** sur une colonne de type texte servant de filtre est :

```
SELECT DISTINCT type_client FROM client;
SELECT DISTINCT categorie FROM vehicule;
```

Exercice « trouve les erreurs » (réponses à la fin de la partie 4) :

```
-- A
CREATE OR REPLACE FUNCTION nb_resa(p_num int) RETURNS int AS
DECLARE v_nb int;
BEGIN
    SELECT count(*) FROM reservation WHERE num_client = p_num;
    RETURN v_nb;
END; LANGUAGE plpgsql;

-- B
CREATE OR REPLACE FUNCTION infos(p_num int) RETURNS record AS $$
DECLARE v_r record;
BEGIN
    SELECT nom_client, pre_client INTO v_r FROM client WHERE num_client = p_num;
    RETURN v_r;
END; $$ LANGUAGE plpgsql;
SELECT infos(100);

-- C
CREATE OR REPLACE FUNCTION liste(p_cat char) RETURNS SETOF record AS $$
DECLARE v_r record;
```

```
BEGIN
    FOR v_r IN SELECT exemple, prix_jour FROM vehicule WHERE categorie = p_cat;
    LOOP
        RETURN v_r;
    END LOOP;
END; $$ LANGUAGE plpgsql;
```

Partie 3 — Méthode au contrôle

3.1 Avant de commencer (5 minutes)

1. Lancer le script de création dans phppgadmin, vérifier que le schéma (`bdd_car_net`) et les tables apparaissent à gauche.
2. Dans la fenêtre SQL, commencer par `SET search_path = bdd_car_net;` (à refaire si vous rouvrez la fenêtre SQL — phppgadmin oublie le `search_path` entre deux exécutions ; sinon préfixez : `bdd_car_net.client`).
3. Lire le MPD en annexe et repérer : les clés étrangères (pour les jointures), les colonnes `NULL` ables (pour les `coalesce`), les types (`TIMESTAMP` vs `DATE`).
4. `SELECT DISTINCT` sur les colonnes de type/état/catégorie.
5. Ouvrir la correction du contrôle d'entraînement et le cours : vous y avez droit, servez-vous-en comme modèles.
6. Sous phppgadmin, **exécutez les tests « cas d'erreur » un par un**, seuls dans la fenêtre SQL : une instruction qui lève une exception annule toutes celles envoyées en même temps qu'elle (y compris un `CREATE FUNCTION` placé juste avant — la fonction n'existera pas et vous aurez `function … does not exist` ensuite).

3.2 Le patron de réponse pour chaque fonction

1. **Écrire d'abord la requête SQL seule** et la tester avec des valeurs en dur. Si la requête est fausse, la fonction le sera aussi.
2. Déterminer le retour avec le tableau 1.6 : combien de lignes ? combien de colonnes ? → `RETURNS …` et `RETURN` / `RETURN NEXT` / `RETURN QUERY`.
3. Écrire la fonction dans cet ordre :
   - vérifications d'existence (`PERFORM` + `IF NOT FOUND THEN RAISE EXCEPTION`), **une par condition demandée**, avec un message explicite contenant la valeur (`%`) ;
   - vérifications de cohérence (`IF date_deb > date_fin THEN RAISE EXCEPTION …`) ;
   - la requête (avec `INTO` ou dans un `FOR`) ;
   - le retour.
4. **Respecter exactement le nom de fonction donné** par le sujet (« Vous devez respecter les noms que je vous donne »).
5. **Fournir les requêtes de test** (« Vous devez me donner les requêtes qui permettent de tester vos fonctions ») : utiliser les valeurs imposées par le sujet, inclure au moins un cas d'erreur par vérification, et écrire le résultat attendu en commentaire (`-- ERREUR: le client 33 n'existe pas`).
6. Commenter chaque bloc (le prof le fait dans toutes ses corrections).

3.3 Grille mentale « quel RETURNS ? »

- « retourne le nombre / la somme / le prix / la moyenne / TRUE ou FALSE » → scalaire.

- « retourne le nom et le prénom de … » (1 individu) → `record`.
- « retourne la liste des … avec leur … » → `SETOF record` + `FOR … LOOP RETURN NEXT`.
- « retourne toutes les informations sur les … » → `SETOF table` + `RETURN QUERY` ou `%ROWTYPE`.
- « met à jour … » sans valeur à renvoyer → `void` (ou renvoyer le nombre de lignes / un booléen si demandé).
- « utilisez un curseur » → déclarer `CURSOR`, `OPEN`, `LOOP FETCH … EXIT WHEN NOT FOUND`, `CLOSE`.

Partie 4 — Exercices d'entraînement (base bdd_car_net)

Toutes ces fonctions ont été testées sur la base du script `create_table_car_net.sql`. Les résultats indiqués sont les vrais résultats. Faites chaque exercice dans phppgadmin **avant** de regarder la correction (section 4.4). Un fichier `solutions_car_net.sql` prêt à coller accompagne ce document.

Rappel du schéma (tables et colonnes utiles) :

- `agence(num_agence, nom_agence, tel_agence, adr_agence, ville_agence, cp_agence, pays_agence)`
- `client(num_client, type_client, civilite, nom_client, pre_client, …, mode_reglement, date_naissance, num_permis, date_permis, pts_bonus_acquis, num_cb)` — `type_client` ∈ { `'Fidéle'`, `'Occassionnel'` }
- `vehicule(code_vehicule, exemple, categorie, prix_jour, nb_portes, nb_passagers, nb_bagages, clim, volume, charge_utile, longueur, largeur, hauteur)` — `categorie` ∈ {A, B, C, K, F}
- `reservation(num_resa, retour_agence, depart_agence, num_client, code_vehicule, date_resa, date_heure_depart, date_heure_retour)` — les deux dernières sont des `TIMESTAMP`
- `proposer(code_vehicule, num_agence)` — quels véhicules propose chaque agence
- `attribuer(code_vehicule, nb_jours, nb_points)` — barème de points par véhicule et par tranche
- `tranche_location(nb_jours)` — tranches 3, 5, 7, 8, 10, 20, 30
- `ouverture(num_agence, num_jour, heure_ouverture, heure_fermeture)` et `jour_semaine(num_jour, libelle_jour)`

4.1 Niveau 1 — fonctions scalaires

Exercice 1. `nb_vehicules()` : renvoie le nombre de véhicules de la flotte. Test : `SELECT nb_vehicules();` → 16.

Exercice 2. `prix_vehicule(code)` : renvoie le prix journalier d'un véhicule. Erreur si le véhicule n'existe pas. Utilisez une variable déclarée avec `%TYPE`. Tests : `(1)` → 33.00 ; `(99)` → erreur. Puis utilisez la fonction dans une requête : les véhicules plus chers que le véhicule 4.

Exercice 3. `nb_jours_resa(num_resa)` : renvoie le nombre de jours d'une réservation (uniquement la partie « jours » de l'intervalle, comme le prof). Erreur si la réservation n'existe pas. Tests : `(78)` → 2 ; `(100)` → 4 ; `(9999)` → erreur.

Exercice 4. `montant_resa(num_resa)` : renvoie le montant d'une réservation = prix journalier × nombre de jours. Réutilisez `nb_jours_resa`. Tests : `(78)` → 262.00 ; `(100)` → 580.00.

Exercice 5. `est_fidele(num_client)` : renvoie `TRUE` si le client est fidèle, `FALSE` sinon ; erreur s'il n'existe pas. Tests : `(100)` → true ; `(103)` → false ; `(1)` → erreur.

Exercice 6. `points_tranche(code_vehicule, nb_jours)` : renvoie le nombre de points qu'attribue ce véhicule pour cette durée, selon la table `attribuer` : on prend la tranche la plus grande qui est ≤ `nb_jours` (pour le véhicule 1 : 3–4 jours → 10 pts, 5–6 → 20, 20–29 → 60). Renvoyer 0 si la durée est inférieure à la première tranche. Erreur si le véhicule n'existe pas ou si `nb_jours` ≤ 0. Tests sur le véhicule 1 : 2 j → 0 ; 3 j → 10 ; 4 j → 10 ; 6 j → 20 ; 25 j → 60 ; 40 j → 100.

4.2 Niveau 2 — plusieurs colonnes, plusieurs lignes

Exercice 7. `info_client(num_client)` : renvoie le nom, le prénom et le type d'un client (une seule ligne, trois colonnes). Erreur s'il n'existe pas. Test : `(100)` → Paul, Troy, Fidéle. Écrivez l'appel correct.

Exercice 8. `vehicules_categorie(cat)` : renvoie le modèle (`exemple`), le prix journalier et la climatisation de tous les véhicules d'une catégorie, triés par prix croissant. Erreur si aucun véhicule de cette catégorie. Test : `('A')` → 4 lignes (opel corsa 33, peugeot 107 51, toyota yaris 78, AUSTIN MINI 135) ; `('Z')` → erreur.

Exercice 9. `agences_pays(pays)` : renvoie **toutes les colonnes** des agences d'un pays, triées par nom. Utilisez `RETURNS SETOF agence` et `RETURN QUERY`. Test : `('US')` → 25 agences, la première est « Ade ».

Exercice 10. `horaires_agence(nom_agence)` : renvoie, pour une agence donnée par son nom (sans tenir compte de la casse), le libellé du jour, l'heure d'ouverture et de fermeture, dans l'ordre des jours. Erreur si l'agence n'existe pas. Test : `('Lavat sud')` → 6 lignes (lundi 07:30–22:00, ...) ; `('Nulle part')` → erreur.

Exercice 11. `meilleur_client()` : renvoie le numéro, le nom, le prénom et le nombre de réservations du client qui a le plus de réservations (une ligne). Test → 124, Rhodes, Silas, 13. Attention au type de `count(*)` dans le `AS (...)`.

Exercice 12. `ca_par_categorie()` : renvoie pour chaque catégorie de véhicule le nombre de réservations et le chiffre d'affaires (somme des montants prix × jours). Utilisez `RETURNS TABLE(...)` et `RETURN QUERY`. Test → A : 57 résas, 43428 ; F : 49 résas, 72072.

4.3 Niveau 3 — curseurs, mises à jour, niveau contrôle

Exercice 13. `reservations_client(num_client)` **avec un curseur paramétré** : renvoie pour un client la liste de ses réservations (numéro, modèle du véhicule, nombre de jours, montant), triées par date de départ. Erreur si le client n'existe pas. Test : `(100)` → 6 lignes, dont (18, opel corsa, 16 j, 528) ; vérifiez que `SELECT sum(montant) FROM reservations_client(100) AS (...)` donne bien 5129 (= `SommePaye(100)` du prof).

Exercice 14. `majorer_prix(cat, pct)` : augmente de `pct` % le prix journalier de tous les véhicules d'une catégorie et renvoie le nombre de véhicules modifiés. Erreur si `pct` n'est pas

dans]0 ; 50] ou si la catégorie n'existe pas. Test : `('A', 10)` → 4, et l'opel corsa passe à 36.30 ; `('A', 80)` → erreur.

Exercice 15. `crediter_points(num_client)` : pour un client **fidèle** (erreur sinon, en réutilisant `est_fidele`), calcule le total des points de toutes ses réservations (avec `points_tranche`), ajoute ce total à `pts_bonus_acquis` (attention au NULL) et renvoie le nombre de points crédités. Test : client 100 a 220 points ; `crediter_points(100)` → 233 ; il a ensuite 453 points. `(103)` → erreur.

Exercice 16. `clients_fideles_sans_points()` **avec un curseur et** `%ROWTYPE` : renvoie toutes les colonnes des clients fidèles dont les points sont NULL ou 0. `RETURNS SETOF client`. Test → 1 client (131, Shields).

Exercice 17 (bonus trigger). Créez un trigger `BEFORE INSERT OR UPDATE` sur `reservation` qui refuse une réservation dont la date de retour est antérieure ou égale à la date de départ, et qui met `date_resa` à la date du jour si elle est NULL. Testez avec un `INSERT` invalide puis un valide.

4.4 Le contrôle d'entraînement, à refaire en conditions réelles (1h30)

Reprenez le sujet `entrainement_controle/Controle fonction.pdf` et refaites les 3 exercices **sans regarder la correction** :

1. `listeVehicule(NomAg)` : modèles et prix journalier des véhicules proposés par une agence. Tests `'Micsot'` (4 lignes), `'Ade'`, `'Micot'` (erreur).
2. `SommePaye(NumeroClient)` : cumul payé par un client. Tests 100 → 5129, 102 → 7402, 103 → 1016, 33 → erreur.
3. `nb_pts(NumeroClient, Num_vehi, date_deb, date_fin)` : points déjà acquis, points attribués, total ; 4 vérifications (client existe, client fidèle, véhicule existe, dates cohérentes). Test `(104, 1, '2020-06-20', '2020-06-30')` → 190, 50, 240.

Indices si vous bloquez : ex. 1 = modèle `SETOF record` (section 1.6) avec jointure `vehicule ⋈ proposer ⋈ agence` ; ex. 2 = scalaire avec `sum(prix_jour * extract(day FROM …))` ; ex. 3 = `record` avec une sous-requête `ORDER BY nb_jours DESC LIMIT 1` sur `attribuer` (c'est exactement `points_tranche` de l'exercice 6, vous pouvez la réutiliser !).

Corrections

Réponses de « trouve les erreurs » (partie 2)

- **A** : (1) pas de `$$` autour du corps ; (2) `SELECT count(*)` sans `INTO v_nb` ; (3) `END; LANGUAGE plpgsql` doit être `END; $$ LANGUAGE plpgsql;`.
- **B** : la fonction est correcte, c'est **l'appel** qui est faux : elle renvoie un `record`. `SELECT infos(100);` ne plante pas mais renvoie une seule colonne contenant `(Paul,Troy)` — inutilisable. Il faut `SELECT * FROM infos(100) AS (nom varchar, prenom varchar);` pour obtenir deux colonnes (et `SELECT * FROM infos(100);` sans `AS` donne l'erreur `a column definition list is required`).
- **C** : (1) `;` après la requête du `FOR` ; (2) `RETURN v_r` au lieu de `RETURN NEXT v_r` — PostgreSQL refuse même la création : `RETURN cannot have a parameter in function returning set` ; (3) pas de `RETURN;` final.

Corrections des exercices 1 à 17

```sql
SET search_path = bdd_car_net;

-- ============ Exercice 1 ============
CREATE OR REPLACE FUNCTION nb_vehicules() RETURNS integer AS $$
DECLARE
    v_nb integer;
BEGIN
    SELECT count(*) INTO v_nb FROM vehicule;
    RETURN v_nb;
END;
$$ LANGUAGE plpgsql;

SELECT nb_vehicules();                    -- 16

-- ============ Exercice 2 ============
CREATE OR REPLACE FUNCTION prix_vehicule(p_code integer) RETURNS numeric AS $$
DECLARE
    v_prix vehicule.prix_jour%TYPE;
BEGIN
    SELECT prix_jour INTO v_prix FROM vehicule WHERE code_vehicule = p_code;
    -- SELECT INTO positionne FOUND : pas besoin d'un PERFORM séparé ici
    IF NOT FOUND THEN
        RAISE EXCEPTION 'le véhicule % n''existe pas', p_code;
    END IF;
    RETURN v_prix;
END;
$$ LANGUAGE plpgsql;

SELECT prix_vehicule(1);                  -- 33.00
SELECT prix_vehicule(99);                 -- ERREUR: le véhicule 99 n'existe pas
SELECT exemple, prix_jour FROM vehicule WHERE prix_jour > prix_vehicule(4);   -- 11
        lignes
```

```sql
-- ============ Exercice 3 ============
CREATE OR REPLACE FUNCTION nb_jours_resa(p_resa integer) RETURNS integer AS $$
DECLARE
    v_nb integer;
BEGIN
    SELECT extract(day FROM (date_heure_retour - date_heure_depart)) INTO v_nb
    FROM reservation WHERE num_resa = p_resa;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'la réservation % n''existe pas', p_resa;
    END IF;
    RETURN v_nb;
END;
$$ LANGUAGE plpgsql;

SELECT nb_jours_resa(78);                  -- 2
SELECT nb_jours_resa(100);                 -- 4
SELECT nb_jours_resa(9999);                -- ERREUR: la réservation 9999 n'existe pas

-- ============ Exercice 4 ============
CREATE OR REPLACE FUNCTION montant_resa(p_resa integer) RETURNS numeric AS $$
DECLARE
    v_prix numeric;
BEGIN
    SELECT prix_jour INTO v_prix
    FROM reservation JOIN vehicule USING (code_vehicule)
    WHERE num_resa = p_resa;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'la réservation % n''existe pas', p_resa;
    END IF;
    RETURN v_prix * nb_jours_resa(p_resa);     -- réutilisation de la fonction précédente
END;
$$ LANGUAGE plpgsql;

SELECT montant_resa(78);                   -- 262.00
SELECT montant_resa(100);                  -- 580.00

-- ============ Exercice 5 ============
CREATE OR REPLACE FUNCTION est_fidele(p_num integer) RETURNS boolean AS $$
DECLARE
    v_type client.type_client%TYPE;
BEGIN
    SELECT type_client INTO v_type FROM client WHERE num_client = p_num;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'le client % n''existe pas', p_num;
    END IF;
    RETURN v_type = 'Fidéle';        -- attention à l'orthographe exacte dans la base
END;
$$ LANGUAGE plpgsql;

SELECT est_fidele(100);                    -- true
SELECT est_fidele(103);                    -- false
SELECT est_fidele(1);                      -- ERREUR: le client 1 n'existe pas

-- ============ Exercice 6 ============
CREATE OR REPLACE FUNCTION points_tranche(p_code integer, p_nb_jours integer) RETURNS
        integer AS $$
DECLARE
    v_pts integer;
BEGIN
    PERFORM * FROM vehicule WHERE code_vehicule = p_code;
```

```
    IF NOT FOUND THEN
        RAISE EXCEPTION 'le véhicule % n''existe pas', p_code;
    END IF;
    IF p_nb_jours <= 0 THEN
        RAISE EXCEPTION 'le nombre de jours doit être positif';
    END IF;
    -- la plus grande tranche inférieure ou égale à la durée
    SELECT nb_points INTO v_pts
    FROM attribuer
    WHERE code_vehicule = p_code AND nb_jours <= p_nb_jours
    ORDER BY nb_jours DESC
    LIMIT 1;
    RETURN coalesce(v_pts, 0);       -- aucune tranche → 0 point
END;
$$ LANGUAGE plpgsql;

SELECT points_tranche(1,2), points_tranche(1,3), points_tranche(1,4),
       points_tranche(1,6), points_tranche(1,25), points_tranche(1,40);   -- 0 10 10 20
        60 100
SELECT points_tranche(1,0);               -- ERREUR: le nombre de jours doit être
        positif

-- ============ Exercice 7 ============
CREATE OR REPLACE FUNCTION info_client(p_num integer) RETURNS record AS $$
DECLARE
    v_info record;
BEGIN
    PERFORM * FROM client WHERE num_client = p_num;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'le client % n''existe pas', p_num;
    END IF;
    SELECT nom_client, pre_client, type_client INTO v_info
    FROM client WHERE num_client = p_num;
    RETURN v_info;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM info_client(100) AS (nom varchar, prenom varchar, type_cli varchar);  --
        Paul | Troy | Fidéle
SELECT * FROM info_client(1)   AS (nom varchar, prenom varchar, type_cli varchar);  --
        ERREUR

-- ============ Exercice 8 ============
CREATE OR REPLACE FUNCTION vehicules_categorie(p_cat char) RETURNS SETOF record AS $$
DECLARE
    v_ligne record;
BEGIN
    PERFORM * FROM vehicule WHERE categorie = p_cat;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'aucun véhicule de catégorie %', p_cat;
    END IF;
    FOR v_ligne IN SELECT exemple, prix_jour, clim
                   FROM vehicule WHERE categorie = p_cat
                   ORDER BY prix_jour
    LOOP
        RETURN NEXT v_ligne;
    END LOOP;
    RETURN;
END;
$$ LANGUAGE plpgsql;
```

```sql
SELECT * FROM vehicules_categorie('A') AS (modele varchar, prix numeric, clim boolean);
        -- 4 lignes
SELECT * FROM vehicules_categorie('Z') AS (modele varchar, prix numeric, clim boolean);
        -- ERREUR

-- ============ Exercice 9 ============
CREATE OR REPLACE FUNCTION agences_pays(p_pays char(2)) RETURNS SETOF agence AS $$
BEGIN
    RETURN QUERY SELECT * FROM agence WHERE pays_agence = p_pays ORDER BY nom_agence;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM agences_pays('US');                              -- 25 lignes, pas de AS
        (...)
SELECT nom_agence, ville_agence FROM agences_pays('US');

-- ============ Exercice 10 ============
CREATE OR REPLACE FUNCTION horaires_agence(p_nom varchar) RETURNS SETOF record AS $$
DECLARE
    v_ligne record;
BEGIN
    PERFORM * FROM agence WHERE lower(nom_agence) = lower(p_nom);
    IF NOT FOUND THEN
        RAISE EXCEPTION 'l''agence % n''existe pas', p_nom;
    END IF;
    FOR v_ligne IN SELECT libelle_jour, heure_ouverture, heure_fermeture
                   FROM agence JOIN ouverture USING (num_agence)
                               JOIN jour_semaine USING (num_jour)
                   WHERE lower(nom_agence) = lower(p_nom)
                   ORDER BY num_jour
    LOOP
        RETURN NEXT v_ligne;
    END LOOP;
    RETURN;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM horaires_agence('Lavat sud')  AS (jour varchar, ouv time, ferm time);  -- 6
        lignes
SELECT * FROM horaires_agence('LAVAT SUD')  AS (jour varchar, ouv time, ferm time);  --
        idem (lower)
SELECT * FROM horaires_agence('Nulle part') AS (jour varchar, ouv time, ferm time);  --
        ERREUR

-- ============ Exercice 11 ============
CREATE OR REPLACE FUNCTION meilleur_client() RETURNS record AS $$
DECLARE
    v_info record;
BEGIN
    SELECT num_client, nom_client, pre_client, count(*) AS nb_resa INTO v_info
    FROM client JOIN reservation USING (num_client)
    GROUP BY num_client, nom_client, pre_client
    ORDER BY nb_resa DESC
    LIMIT 1;
    RETURN v_info;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM meilleur_client() AS (num int, nom varchar, prenom varchar, nb bigint);  --
        124 Rhodes Silas 13
```

```sql
-- ============ Exercice 12 ============
CREATE OR REPLACE FUNCTION ca_par_categorie()
RETURNS TABLE (categorie char, nb_resa bigint, chiffre_affaires numeric) AS $$
BEGIN
    RETURN QUERY
        SELECT v.categorie, count(*),
               sum(prix_jour * extract(day FROM (date_heure_retour - date_heure_depart)))
        FROM reservation r JOIN vehicule v USING (code_vehicule)
        GROUP BY v.categorie
        ORDER BY v.categorie;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM ca_par_categorie();          -- A 57 43428 | B 35 36354 | C 26 26227 | F 49
        72072 | K 33 33763

-- ============ Exercice 13 (curseur paramétré) ============
CREATE OR REPLACE FUNCTION reservations_client(p_num integer) RETURNS SETOF record AS $$
DECLARE
    v_ligne record;
    curs_resa CURSOR (p_cli integer) FOR
        SELECT num_resa, exemple,
               extract(day FROM (date_heure_retour - date_heure_depart))::int AS
        nb_jours,
               prix_jour * extract(day FROM (date_heure_retour - date_heure_depart)) AS
        montant
        FROM reservation JOIN vehicule USING (code_vehicule)
        WHERE num_client = p_cli
        ORDER BY date_heure_depart;
BEGIN
    PERFORM * FROM client WHERE num_client = p_num;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'le client % n''existe pas', p_num;
    END IF;
    OPEN curs_resa(p_num);
    LOOP
        FETCH curs_resa INTO v_ligne;
        EXIT WHEN NOT FOUND;
        RETURN NEXT v_ligne;
    END LOOP;
    CLOSE curs_resa;
    RETURN;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM reservations_client(100) AS (num_resa int, vehicule varchar, nb_jours int,
        montant numeric);
-- 18 opel corsa 16 528 | 144 toyota yaris 10 780 | 178 citroen jumper 18 2610 | 182 ...
        | 358 ... | 210 ...
SELECT sum(montant) FROM reservations_client(100) AS (num_resa int, vehicule varchar,
        nb_jours int, montant numeric);
-- 5129  (= SommePaye(100))

-- ============ Exercice 14 ============
CREATE OR REPLACE FUNCTION majorer_prix(p_cat char, p_pct numeric) RETURNS integer AS $$
DECLARE
    v_nb integer;
BEGIN
    IF p_pct <= 0 OR p_pct > 50 THEN
        RAISE EXCEPTION 'pourcentage % invalide (doit être entre 0 et 50)', p_pct;
    END IF;
```

```
    PERFORM * FROM vehicule WHERE categorie = p_cat;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'aucun véhicule de catégorie %', p_cat;
    END IF;
    UPDATE vehicule SET prix_jour = round(prix_jour * (1 + p_pct/100), 2)
    WHERE categorie = p_cat;
    GET DIAGNOSTICS v_nb = ROW_COUNT;    -- nombre de lignes modifiées par l'UPDATE
    RETURN v_nb;
END;
$$ LANGUAGE plpgsql;

SELECT majorer_prix('A', 10);              -- 4
SELECT code_vehicule, exemple, prix_jour FROM vehicule WHERE categorie = 'A';   -- opel
        corsa 36.30 ...
SELECT majorer_prix('A', 80);              -- ERREUR: pourcentage 80 invalide

-- ============ Exercice 15 ============
CREATE OR REPLACE FUNCTION crediter_points(p_num integer) RETURNS integer AS $$
DECLARE
    v_resa  record;
    v_total integer := 0;
BEGIN
    -- est_fidele lève déjà l'exception si le client n'existe pas
    IF NOT est_fidele(p_num) THEN
        RAISE EXCEPTION 'le client % n''est pas Fidéle, pas de points', p_num;
    END IF;
    FOR v_resa IN SELECT code_vehicule,
                         extract(day FROM (date_heure_retour - date_heure_depart))::int
        AS nbj
                  FROM reservation WHERE num_client = p_num
    LOOP
        v_total := v_total + points_tranche(v_resa.code_vehicule, v_resa.nbj);
    END LOOP;
    UPDATE client SET pts_bonus_acquis = coalesce(pts_bonus_acquis, 0) + v_total
    WHERE num_client = p_num;
    RETURN v_total;
END;
$$ LANGUAGE plpgsql;

SELECT pts_bonus_acquis FROM client WHERE num_client = 100;   -- 220
SELECT crediter_points(100);                                   -- 233
SELECT pts_bonus_acquis FROM client WHERE num_client = 100;   -- 453
SELECT crediter_points(103);                                   -- ERREUR: le client 103
        n'est pas Fidéle

-- ============ Exercice 16 (curseur + %ROWTYPE) ============
CREATE OR REPLACE FUNCTION clients_fideles_sans_points() RETURNS SETOF client AS $$
DECLARE
    v_cli client%ROWTYPE;
    curs_cli CURSOR FOR
        SELECT * FROM client
        WHERE type_client = 'Fidéle' AND coalesce(pts_bonus_acquis, 0) = 0
        ORDER BY nom_client;
BEGIN
    OPEN curs_cli;
    LOOP
        FETCH curs_cli INTO v_cli;
        EXIT WHEN NOT FOUND;
        RETURN NEXT v_cli;
    END LOOP;
    CLOSE curs_cli;
```

```
    RETURN;
END;
$$ LANGUAGE plpgsql;

SELECT num_client, nom_client, pts_bonus_acquis FROM clients_fideles_sans_points();   --
        131 Shields NULL

-- ============ Exercice 17 (trigger) ============
CREATE OR REPLACE FUNCTION verif_dates_resa() RETURNS trigger AS $$
BEGIN
    IF NEW.date_heure_retour <= NEW.date_heure_depart THEN
        RAISE EXCEPTION 'retour (%) doit être après départ (%)', NEW.date_heure_retour,
        NEW.date_heure_depart;
    END IF;
    IF NEW.date_resa IS NULL THEN
        NEW.date_resa := current_date;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_verif_dates_resa BEFORE INSERT OR UPDATE ON reservation
FOR EACH ROW EXECUTE PROCEDURE verif_dates_resa();

-- refusé
INSERT INTO reservation (retour_agence, depart_agence, num_client, code_vehicule,
        date_resa, date_heure_depart, date_heure_retour)
VALUES (1007, 1007, 100, 1, '2024-01-01', '2024-01-10 08:00', '2024-01-05 08:00');
-- accepté
INSERT INTO reservation (retour_agence, depart_agence, num_client, code_vehicule,
        date_resa, date_heure_depart, date_heure_retour)
VALUES (1007, 1007, 100, 1, '2024-01-01', '2024-01-05 08:00', '2024-01-10 08:00');
```

Correction du contrôle d'entraînement (version du prof, commentée)

```
-- Exercice 1 : SETOF record, jointure vehicule-proposer-agence, lower() pour la casse
CREATE OR REPLACE FUNCTION listeVehicule(p_nom_ag varchar(45)) RETURNS setof record AS $$
DECLARE
    v_ligne record;
BEGIN
    PERFORM nom_agence FROM agence WHERE lower(nom_agence) = lower(p_nom_ag);
    IF NOT FOUND THEN
        RAISE EXCEPTION 'l''agence % n''existe pas', p_nom_ag;
    END IF;
    FOR v_ligne IN SELECT exemple, prix_jour
                   FROM vehicule INNER JOIN proposer USING (code_vehicule)
                                 INNER JOIN agence USING (num_agence)
                   WHERE lower(nom_agence) = lower(p_nom_ag)
    LOOP
        RETURN NEXT v_ligne;
    END LOOP;
    RETURN;
END
$$ LANGUAGE plpgsql;

SELECT * FROM listeVehicule('Micsot') AS (nom_vehi varchar, px numeric);   -- 4 lignes
SELECT * FROM listeVehicule('Ade')    AS (nom_vehi varchar, px numeric);
```

```sql
SELECT * FROM listeVehicule('Micot')  AS (nom_vehi varchar, px numeric);   -- ERREUR:
        l'agence Micot n'existe pas

-- Exercice 2 : scalaire, extract(day FROM intervalle)
CREATE OR REPLACE FUNCTION SommePaye(p_numCli int) RETURNS real AS $$
DECLARE
    somme real;
BEGIN
    PERFORM num_client FROM client WHERE num_client = p_numCli;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'le client % n''existe pas', p_numCli;
    END IF;
    SELECT sum(prix_jour * extract(day FROM (date_heure_retour - date_heure_depart)))
    INTO somme
    FROM reservation INNER JOIN vehicule USING (code_vehicule)
    WHERE num_client = p_numCli;
    RETURN somme;
END
$$ LANGUAGE plpgsql;

SELECT SommePaye(100);   -- 5129
SELECT SommePaye(102);   -- 7402
SELECT SommePaye(103);   -- 1016
SELECT SommePaye(33);    -- ERREUR: le client 33 n'existe pas

-- Exercice 3 : record, 4 vérifications, sous-requête ORDER BY nb_jours DESC LIMIT 1
CREATE OR REPLACE FUNCTION nb_pts(NumeroClient int, Num_vehi int, date_deb date, date_fin
        date)
RETURNS record AS $$
DECLARE
    info record;
BEGIN
    PERFORM num_client FROM client WHERE num_client = NumeroClient;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'le client % n''existe pas', NumeroClient;
    END IF;
    PERFORM num_client FROM client WHERE type_client = 'Fidéle' AND num_client =
        NumeroClient;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'le client % n''est pas Fidéle', NumeroClient;
    END IF;
    PERFORM code_vehicule FROM vehicule WHERE code_vehicule = Num_vehi;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'le véhicule % n''existe pas', Num_vehi;
    END IF;
    IF date_deb > date_fin THEN
        RAISE EXCEPTION 'la date de début ne peut être plus grande que la date de fin';
    END IF;
    SELECT coalesce(pts_bonus_acquis, 0) AS pts_deja_acquis,
           (SELECT nb_points FROM attribuer
            WHERE nb_jours <= date_fin - date_deb AND code_vehicule = Num_vehi
            ORDER BY nb_jours DESC LIMIT 1) AS nb_pts_attribuer,
           coalesce(pts_bonus_acquis, 0) +
           (SELECT nb_points FROM attribuer
            WHERE nb_jours <= date_fin - date_deb AND code_vehicule = Num_vehi
            ORDER BY nb_jours DESC LIMIT 1) AS pts_total
    INTO info
    FROM client WHERE num_client = NumeroClient;
    RETURN info;
END;
$$ LANGUAGE plpgsql;
```

```sql
SELECT * FROM nb_pts(300, 1, '2020-06-30', '2020-06-20') AS (deja int, attribues int,
        total int);  -- client 300 n'existe pas
SELECT * FROM nb_pts(100, 78, '2020-06-30', '2020-06-20') AS (deja int, attribues int,
        total int); -- véhicule 78 n'existe pas
SELECT * FROM nb_pts(103, 1, '2020-06-20', '2020-06-30') AS (deja int, attribues int,
        total int);  -- client 103 n'est pas Fidéle
SELECT * FROM nb_pts(104, 1, '2020-06-30', '2020-06-20') AS (deja int, attribues int,
        total int);  -- dates incohérentes
SELECT * FROM nb_pts(104, 1, '2020-06-20', '2020-06-30') AS (deja int, attribues int,
        total int);  -- 190 | 50 | 240
```

Variante plus lisible de l'exercice 3 en réutilisant `points_tranche` (exercice 6) — à utiliser si vous avez déjà écrit la fonction :

```sql
    SELECT coalesce(pts_bonus_acquis, 0),
           points_tranche(Num_vehi, date_fin - date_deb),
           coalesce(pts_bonus_acquis, 0) + points_tranche(Num_vehi, date_fin - date_deb)
    INTO info
    FROM client WHERE num_client = NumeroClient;
```

Mémo d’une page (à relire juste avant d’entrer)

```
SET search_path = bdd_car_net;                          -- toujours en premier
SELECT DISTINCT type_client FROM client;                -- vérifier les libellés exacts
        ('Fidéle')

CREATE OR REPLACE FUNCTION f(p_x int) RETURNS SETOF record AS $$
DECLARE
    v_r record;
BEGIN
    PERFORM * FROM t WHERE id = p_x;                    -- 1. existence
    IF NOT FOUND THEN RAISE EXCEPTION 'le ... % n''existe pas', p_x; END IF;
    FOR v_r IN SELECT a, b FROM t WHERE id = p_x        -- 2. requête (pas de ; ici)
    LOOP
        RETURN NEXT v_r;                                -- 3. retour ligne par ligne
    END LOOP;
    RETURN;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM f(1) AS (a varchar, b numeric);           -- 4. tests, avec un cas d'erreur
```

- 1 valeur → `RETURNS type` + `RETURN v;` + `SELECT f();`
- 1 ligne N colonnes → `RETURNS record` + `SELECT * FROM f() AS (...)`
- N lignes N colonnes → `RETURNS SETOF record` + `RETURN NEXT` + `RETURN;` + `SELECT * FROM f() AS (...)`
- lignes d’une table → `RETURNS SETOF table` + `RETURN QUERY SELECT …;` + `SELECT * FROM f();`
- curseur → `c CURSOR (k int) FOR SELECT …;` / `OPEN c(x);` / `LOOP FETCH c INTO v; EXIT WHEN NOT FOUND; … END LOOP;` / `CLOSE c;`
- jours entre deux timestamps → `extract(day FROM (fin - debut))` ; entre deux dates → `fin - debut`
- NULL → `coalesce(x, 0)` ; apostrophe → `''` ; changement de type de retour → `DROP FUNCTION` d’abord.